

PawsAndWait — Road Test Test Plan & Procedure

Raspberry Pi Compute Module 5 Development Kit

Project: AI-Powered Smart Pet Feeder with Animal Recognition & Pose Detection

Product Under Test	Raspberry Pi CM5 Development Kit (CM5104032 — 4GB RAM, 32GB eMMC, Wi-Fi)
Test Duration	6 weeks from receipt of hardware
Primary Skills	AI/ML, Web Development, Python, Edge Computing
Personal Motivation	Dog owner (2 dogs requiring obedience training), cat owner (cat food regularly stolen by birds), interest in practical AI at the edge

1. Project Overview

PawsAndWait builds a complete, production-quality AI-powered smart pet feeder system using the Raspberry Pi CM5 as the embedded compute platform. The project addresses two real-world problems:

1. feeding dogs remotely - whilst still commanding them to sit and wait
2. feeding a cat whose outdoor food bowl is raided by birds.

The system demonstrates the CM5's suitability as an embedded AI product platform by exercising its most distinctive hardware capabilities simultaneously:

- dual **MIPI CSI** camera inputs,
- the **M.2 PCIe** slot (hosting a Hailo-8L AI accelerator),
- **GPIO** for motor and servo actuation,
- **I2C** for sensor communication,
- the **eMMC/NVMe** storage for a local database and web server.

The road test will produce benchmarked, reproducible data comparing **CM5 CPU-only** inference against **Hailo-accelerated inference** — a direct test of the hardware combination that the CM5 IO board's M.2 slot was designed for.

2. Problem Statement

2.1 Dog Feeding Station

We are an active family with 4 kids and often away here and there for a night. The dogs can be left at home for 2 nights but one is a grazer and the other is an “all you can eat” so they need to be fed in a controlled way so that both get a fair share of food. **PawsAndWait** will build a feeder that makes sure both dogs are sitting and ready to eat before giving them the command to eat.

2.2 Cat Feeding Station

The outdoor cat bowl is regularly accessed by birds (primarily pest Indian Mynas) if the cat has not finished all her food and wants to “graze” on her food throughout the day. This results in wasted food and the cat going hungry. The system will use real-time species detection to keep the bowl lid closed when non-cat animals are detected, only opening when the cat is confirmed present within the feeding zone.

3. Hardware Architecture

3.1 Core Hardware — Provided by element14

Component	Specification	Role in Project
CM5 Module	CM5104032: BCM2712, 4GB LPDDR4, 32GB eMMC, Wi-Fi/BT	Main CPU, OS, decision logic, web server
CM5 IO Board	Dual CSI/DSI, M.2 slot, 40-pin GPIO, Gigabit Ethernet, USB 3.0	Carrier board, exposes all peripherals
IO Case + Fan	Metal enclosure with integrated cooling fan	Thermal management for sustained AI load
27W USB-C PSU	Official Raspberry Pi 27W power supply	Powers CM5 and all peripherals
Antenna Kit	Dual-band Wi-Fi antenna	Reliable wireless for remote dashboard access

3.2 Additional Hardware — Self-Purchased

Component	Approx. Cost (AUD)	Purpose
Raspberry Pi AI HAT+ (13 TOPS, Hailo-8L)	~\$115	AI inference acceleration via M.2 slot

Component	Approx. Cost (AUD)	Purpose
Raspberry Pi Camera Module 3 x2	~\$80 each	One per feeding station, MIPI CSI connection
Stepper motor + DRV8825 driver	~\$25	Dog food auger drive
SG90 servo motor	~\$8	Cat bowl lid open/close mechanism
HX711 + 5kg load cell	~\$15	Food portion weight measurement
MAX98357 I2S amplifier + speaker	~\$20	Voice command audio output for dogs
Miscellaneous (wiring, mounts, relay)	~\$30	Assembly and integration

3.3 System Architecture Summary

Camera 1 (dog station) and Camera 2 (cat station) connect to the CM5 IO board's two combo MIPI CSI/DSI ports. The Hailo-8L AI accelerator sits in the M.2 slot on the IO board, connected via PCIe. The CM5 CPU runs the decision logic, web dashboard (FastAPI + WebRTC), and GPIO motor control. A local SQLite database on the 32GB of eMMC logs all feeding events, detection results, and benchmarks.

4. AI Inference Pipeline

4.1 Model Stack

Model	Framework	Task	Running On
YOLOv8s — species detection	Ultralytics / Hailo HEF	Classify: cat / dog / bird / other + bounding box	Hailo-8L
YOLOv8-pose — dog pose	Ultralytics dog-pose dataset (24 keypoints)	Detect sitting pose from hip/spine keypoints	Hailo-8L
Sit classifier	Lightweight scikit-learn on keypoint output	Convert keypoint geometry to sit/stand boolean	CM5 CPU
Cat proximity check	Geometric zone check on bounding box	Confirm cat within 40cm of bowl before opening	CM5 CPU

4.2 Dog Sit Detection Logic

The dog pose model outputs 24 keypoints per detected dog, each with (x, y, visibility) coordinates. The sit classifier uses the following geometric rules:

- Hip keypoints (left_hip, right_hip) are within 30% of frame height from the bottom of the bounding box
- Spine angle (nose-to-hip vector) is within 45 degrees of vertical
- Both front paws are visible and positioned below the shoulder keypoints
- Confidence score for each keypoint exceeds 0.6

The dog must maintain a confirmed sitting pose for 10.0 second continuously before the food dispenser is activated. If the dog stands during this window, the timer resets and a second voice command plays.

4.3 Cat Feeder Logic

The cat bowl lid remains closed (servo at 0 degrees) at all times. The lid opens only when:

- A cat bounding box is detected with confidence > 0.75
- The bounding box centre falls within a predefined proximity zone (configurable in the web dashboard)
- No bird bounding box is simultaneously present within the same zone

The lid closes again 30 seconds after the cat's bounding box exits the zone, or immediately if a bird is detected while the lid is open.

5. Test Procedure

Week 1: Hardware Setup & Baseline

Phase 1A: CM5 Dev Kit Setup

1. Flash Raspberry Pi OS (64-bit) to CM5 eMMC using rpiboot and Raspberry Pi Imager
2. Boot CM5, confirm all IO board ports functional: HDMI, USB 3.0, Ethernet, GPIO header
3. Connect Wi-Fi antenna, verify dual-band connectivity, measure signal strength
4. Enable SSH and VNC, confirm remote access over local network

Phase 1B: Camera Setup

1. Connect Camera Module 3 to CSI port 0 (dog station), verify libcamera detection
2. Connect Camera Module 3 to CSI port 1 (cat station), verify simultaneous dual-camera capture

3. Benchmark: measure frame capture latency and CPU usage for dual 1080p streams
4. Test MIPI CSI bandwidth: confirm both cameras can stream simultaneously at 30fps without frame drops

Phase 1C: Baseline Inference Benchmark (CPU Only)

1. Install Ultralytics YOLOv8, run YOLOv8s object detection on live camera feed
 2. Measure and record: FPS, CPU utilisation, RAM usage, CPU temperature under sustained load
 3. Run for 10 minutes continuously, log thermal throttling events if any
 4. Record baseline: CPU-only inference FPS with single camera, then dual camera
-

Week 2: AI Accelerator Integration

Phase 2A: Hailo-8L Installation

1. Install Hailo-8L AI HAT+ module into CM5 IO board M.2 slot
2. Install Hailo software stack: `sudo apt install hailo-all`
3. Verify Hailo chip detection: `hailortcli fw-control identify`
4. Run Hailo diagnostics, confirm PCIe link and firmware version

Phase 2B: Accelerated Inference Benchmark

1. Compile YOLOv8s to Hailo HEF format using Hailo Dataflow Compiler
2. Run accelerated object detection on live camera feed
3. Measure and record: FPS, CPU utilisation, RAM usage, temperature — compare directly to Phase 1C baseline
4. Run species classification (cat/dog/bird) on test image set of 200 images, record accuracy
5. Test dual-camera inference: both cameras running simultaneously through Hailo pipeline

Phase 2C: Dog Pose Model

1. Download Ultralytics dog-pose dataset (6,773 training images, 24 key-points)
 2. Fine-tune YOLOv8-pose on dog-pose dataset for sit/stand classification
 3. Compile fine-tuned model to HEF format for Hailo-8L deployment
 4. Test sit detection accuracy: 50 sit trials and 50 stand trials with own dogs, record true/false positive rates
 5. Measure time from dog sitting to dispenser activation: target under 1.5 seconds
-

Week 3: GPIO & Hardware Actuation

Phase 3A: Dog Feeder Motor

1. Wire DRV8825 stepper driver to CM5 GPIO pins, connect auger motor
2. Write GPIO control script, test motor direction, speed, and step count for portion control
3. Calibrate portion size: measure food weight dispensed per motor revolution using load cell
4. Test GPIO latency: measure time from Python GPIO.output() call to physical motor start
5. Run 50 dispense cycles, verify consistent portion weight (target: within 10% of set portion)

Phase 3B: Cat Feeder Servo

1. Wire SG90 servo to CM5 GPIO PWM pin, test open/close positions
2. Test servo response time: measure time from detection event to lid open (target: under 500ms)
3. Run 100 open/close cycles, verify servo reliability and position repeatability

Phase 3C: Audio System

1. Wire MAX98357 I2S amplifier to CM5 I2S GPIO pins
2. Test audio output: play sit command audio clip
3. Test audio latency: measure time from detection event to audio playback start

Phase 3D: Load Cell & Weight Sensing

1. Wire HX711 to I2C GPIO pins, configure load cell calibration
2. Test weight reading accuracy vs known calibration weights
3. Integrate weight confirmation into dispense cycle: dispense only confirmed when weight delta detected

Week 4: Integration & Full System Test

Phase 4A: Dog Station Integration

1. Integrate full pipeline: Camera 1 → Hailo species detection → pose model → sit classifier → GPIO motor + audio
2. Run 30 live feeding trials with own dogs, varying approach angle and lighting conditions
3. Record: detection accuracy, sit detection rate, false positives, time to dispense, portion weight
4. Test edge cases: two dogs present simultaneously, dog partially out of frame, low light conditions

Phase 4B: Cat Station Integration

1. Integrate full pipeline: Camera 2 → Hailo species detection → proximity zone check → servo lid control
2. Run 20 live cat feeding trials, record lid open accuracy
3. Test bird rejection: present bird images/toy to camera, confirm lid remains closed
4. Test simultaneous presence of cat and bird, confirm system prioritises lid-closed state

Phase 4C: Dual Station Simultaneous Operation

1. Run both stations simultaneously for 2-hour continuous test
 2. Monitor: CPU usage, Hailo utilisation, RAM, temperature, GPIO timing, network latency
 3. Verify no resource contention between the two camera pipelines
 4. Log all events to SQLite database, verify data integrity
-

Week 5: Web Dashboard & Remote Access

Phase 5A: Dashboard Development

1. Build FastAPI web server on CM5: live MJPEG streams from both cameras
2. Implement feeding schedule configuration: set meal times, portion sizes per station
3. Build feeding log view: timestamp, species detected, pose result, portion dispensed, weight confirmed
4. Add manual override controls: dispense now, open/close lid, play audio command

Phase 5B: Remote Access Testing

1. Configure Tailscale VPN on CM5 for secure remote access
 2. Test dashboard access from mobile browser over 4G (simulating remote monitoring)
 3. Measure dashboard load time and stream latency over remote connection
 4. Test push notifications: configure email/webhook alert when bird is detected or dog fails to sit after 3 attempts
-

Week 6: Final Benchmarking & Write-up

Phase 6A: Comprehensive Benchmarks

Benchmark	Method	Target
CPU-only inference FPS	YOLOv8s, single camera, 10 min sustained	Record baseline
Hailo inference FPS	YOLOv8s HEF, single camera, 10 min sustained	> 60 FPS
Dual-camera Hailo FPS	Both cameras, 10 min sustained	> 30 FPS each
CPU usage (Hailo active)	htop during Hailo inference	< 40% CPU
GPIO trigger latency	Oscilloscope / logic analyser on motor pin	< 10ms
Time-to-dispense (sit confirmed)	Detection to motor start	< 1.5 seconds
Lid response time	Cat detected to servo open	< 500ms
Bird rejection accuracy	50 bird image presentations	> 98% rejection rate
Dog sit detection accuracy	50 sit trials	> 90% true positive
Portion weight consistency	50 dispense cycles	Within 10% of target
Thermal throttling	2 hour full load	No throttling events
eMMC write endurance	Log write rate over 6 weeks	Record for analysis

Phase 6B: Review Documentation The final element14 community review will include:

- Unboxing and hardware quality assessment of the CM5 Dev Kit
- Step-by-step setup guide for Hailo-8L on CM5 IO board M.2 slot (original documentation gap)
- All benchmark results with graphs comparing CPU-only vs Hailo-accelerated performance
- Full project build log with photos and video of both feeding stations in operation
- Code repository (GitHub) with all Python, model training, and web dashboard code
- Honest assessment of CM5 as an embedded AI product platform vs. regular Pi 5
- Bill of materials with final costs and sourcing links

6. Risk Register

Risk	Likelihood	Mitigation
Hailo-8L M.2 compatibility on CM5 IO board vs Pi 5 HAT+ form factor	Medium	Confirm with CM5 IO board docs and element14 community pre-purchase; document findings in review
Dog pose model accuracy insufficient for sit detection	Low	Fall back to simpler hip-height threshold heuristic; both approaches documented as benchmark
Bird species not in COCO training set (Australian birds)	Low	COCO includes generic ‘bird’ class which covers all species; proximity zone as secondary filter
Thermal throttling under dual-camera + Hailo load	Low	IO Case fan provides active cooling; test with/without fan, document thermal performance
GPIO latency too high for responsive motor control	Low	Use pigpio daemon for hardware-timed GPIO; benchmark and document
eMMC wear from high-frequency logging	Low	Could look at direct log writes to NVMe SSD; eMMC used for OS only

7. Qualifications & Relevant Experience

- **AI & Web Development (Professional):** Expertise in AI integration, web application development, and API design. Experience deploying machine learning models in production web environments translates directly to building the inference pipeline and web dashboard for this project.
- **Electronics (Hobbyist):** Practical experience with GPIO, servo control, sensor integration, and microcontroller projects. Sufficient to implement the motor control, servo, and I2C sensor components of this build.
- **Personal Motivation (Dogs & Cat):** As the owner of two dogs and a cat with an outdoor feeding station regularly raided by birds, this project solves real problems in the applicant’s household. Testing will be conducted

with actual animals in real conditions, producing authentic performance data rather than controlled laboratory results.

This road test is not a proof-of-concept — it is a complete, functional system that will remain in daily use after the review is published, ensuring the review reflects genuine long-term usability rather than a short-term demonstration.

8. Deliverables & Timeline

Week	Deliverable
Week 1	Unboxing post + hardware quality review + camera and baseline CPU benchmark results
Week 2	Hailo-8L setup guide + accelerated inference benchmark results vs CPU baseline
Week 3	GPIO motor control guide + dog pose model training walkthrough + actuation test results
Week 4	Full integration test results + live feeding video demonstrations
Week 5	Web dashboard code + remote access setup guide + dashboard screenshots
Week 6	Final comprehensive review: all benchmarks, conclusions, GitHub repo, bill of materials

All posts will be published to the element14 community road test group with full code, measurements, and photographs. The project GitHub repository will be made public upon completion of the review period.

current submission lives as an idea here: - https://github.com/saramic/learning/blob/master/ideation/2026_element14_road_test_PI_compute_5/SUBMISSION.md